PARMCO Checkpoint 2

AI Interaction Log - October 20, 2026

15 user prompts and corresponding assistant answers from the available conversation. User prompts are transcribed verbatim (except the screenshot is noted as an attachment). Assistant answers are condensed, faithful summaries rather than exact verbatim copies; some visual previews and block diagrams are described in words. For a verbatim archive, also retain/export the original ChatGPT conversation.

Exchange 1

User prompt (verbatim)

> OK now beginning checkpoint 2. I am doing the inventor interface and the bluetooth connection between the pi and the app

Assistant answer (condensed transcription)

Perfect! So for Checkpoint 2, you're taking responsibility for the MIT App Inventor interface and the Bluetooth connection between your Android phone and Raspberry Pi 4.

What you need working by October 29: Android app using MIT App Inventor; three interface controls; Bluetooth initiated from app; Pi receives and displays commands; Pi responds to phone; disconnect and reconnect. The phone does not need to control the motor until CP3.

Suggested initial app: PARMCO_Controller, with Connect Bluetooth, Disconnect, START, STOP, CHANGE DIRECTION, status label, and Pi response label. Initial commands: START, STOP, DIR:CW or DIR:CCW.

Suggested Designer components: ListPicker_Connect, Button_Disconnect, Button_Start, Button_Stop, Button_Direction, Label_Status, Label_Response, BluetoothClient1, Clock1.

Bluetooth Classic architecture: Android App Inventor BluetoothClient connects to a compatible Raspberry Pi Bluetooth server; Pi prints received commands and sends acknowledgments (such as ACK:START). Pairing alone does not establish an app-level server connection.

Start with Designer, then Blocks and Raspberry Pi Bluetooth setup. Asked whether the project had been created.

Exchange 2

User prompt (verbatim)

> Its been opened and teh project created. Idk where the blue tooth stuff is cause that was the first thing i was gonna add

Assistant answer (condensed transcription)

In MIT App Inventor Designer, open Palette > Connectivity and drag BluetoothClient onto Screen1. It appears under Non-visible components as BluetoothClient1. Then add a User Interface > ListPicker, rename it ListPicker_Connect, and set its Text to Connect Bluetooth. The ListPicker lets the user select a paired Bluetooth device. Asked to add those two components before moving to Blocks.

Exchange 3

User prompt (verbatim)

> k those are there

Assistant answer (condensed transcription)

Open Blocks. Add when ListPicker_Connect.BeforePicking do. Inside it, use set ListPicker_Connect.Elements to BluetoothClient1.AddressesAndNames. This populates the device selection list with already-paired Bluetooth devices. The app does not connect yet; the AfterPicking event will handle that. Asked to build these blocks first.

Exchange 4

User prompt (verbatim)

> k thats set up

Assistant answer (condensed transcription)

Add when ListPicker_Connect.AfterPicking do. Inside an if/then/else, call BluetoothClient1.Connect with ListPicker_Connect.Selection as the address. Because Connect returns true or false, use it as the if condition. Add Label_Status in Designer (default Disconnected, red). If connected, set Label_Status.Text to Connected and green; otherwise Connection Failed and red. The Raspberry Pi must be paired and running a compatible Bluetooth Classic server for connection to succeed.

Exchange 5

User prompt (verbatim)

> OK the blocks are set up for that. Lets go ahead and add the start and stop to it before setting up the bluetooth server as I dont have access to the pi right now

Assistant answer (condensed transcription)

Add Button_Start and Button_Stop in Designer. Names Button_Start and Button_Stop; text START and STOP; suggested green and red backgrounds.

In Blocks, each button's Click event checks BluetoothClient1.IsConnected. If true, call BluetoothClient1.SendText with START or STOP, initially planned with a newline terminator. If false, set Label_Status.Text to Bluetooth Not Connected. Suggested Label_LastCommand, default Last Command: None, updated after sending to show START or STOP.

The assistant first proposed a Text > join block combining the command with a newline block and said actual Bluetooth transmission could only be tested once the Pi server was available. Next planned feature: Direction.

Exchange 6

User prompt (verbatim)

> I dont see a newline block

Assistant answer (condensed transcription)

The assistant proposed two alternatives: put an actual line break after START in a multiline text block, or join START with a character created from code point 10 (ASCII line feed). It warned that typing the literal characters backslash and n would not necessarily produce a newline, and recommended the code-point approach.

Exchange 7

User prompt (verbatim)

> like this? [screenshot attached]

Assistant answer (condensed transcription)

The assistant examined the screenshot and said that a pink text block containing '10' would append the characters 1 and 0, not a newline. It recommended keeping join and START, replacing the text '10' with a character-from-code-point block taking numeric 10, and using uppercase START. This advice was later revised when the user could not find that block.

Exchange 8

User prompt (verbatim)

> character from codepoint doesnt exist anywhere i can see it

Assistant answer (condensed transcription)

The assistant acknowledged that the named block was incorrect, mentioned the Text > unicode block, and advised temporarily sending simple START and STOP text without a delimiter until the Raspberry Pi server was configured. It noted that message framing would need to be addressed later.

Exchange 9

User prompt (verbatim)

> k so now how do i get this on my phone to test as it currently is on my computer

Assistant answer (condensed transcription)

Install the MIT AI2 Companion Android app. Connect phone and computer to the same Wi-Fi. In the App Inventor project on the computer, choose Connect > AI Companion, then scan the displayed QR code with the Companion app. The project should load on the phone and update as the designer changes.

Suggested offline tests: START and STOP should show Bluetooth Not Connected; Connect Bluetooth should list paired devices if permissions allow; selecting a device may fail until the Pi server is ready. For standalone installation later, use Build > Android App (.apk). Asked whether the interface appeared.

Exchange 10

User prompt (verbatim)

> it does

Assistant answer (condensed transcription)

The assistant confirmed the Android interface was running and asked for three checks: START changes status to Bluetooth Not Connected; STOP does the same; Connect Bluetooth opens the paired-device list. It proposed adding Direction after those checks.

Exchange 11

User prompt (verbatim)

> yup

Assistant answer (condensed transcription)

Add Button_Direction (initial text Direction: CW) and global variable Direction initialized to CW. On Button_Direction.Click, toggle the variable CW <-> CCW, update the button text, and, if BluetoothClient1.IsConnected, call SendText with join DIR: and the direction. Otherwise show Bluetooth Not Connected. The button should visibly toggle even without Bluetooth. Message framing would be finalized during Pi setup.

Exchange 12

User prompt (verbatim)

> yup thats working

Assistant answer (condensed transcription)

The assistant confirmed all three CP2 command controls worked locally and guided adding Button_Disconnect. In Designer: Button_Disconnect, text Disconnect Bluetooth, gray background. In Blocks: when Button_Disconnect.Click, if BluetoothClient1.IsConnected then call BluetoothClient1.Disconnect; then set Label_Status.Text to Disconnected and TextColor to red.

It summarized local status: Android Companion working; START, STOP, Direction toggle and paired-device selection working locally; Disconnect next; receiving Pi responses and actual Bluetooth communication not yet tested.

Exchange 13

User prompt (verbatim)

> That works, so now pi messaging

Assistant answer (condensed transcription)

Add Label_PiResponse with text Pi Response: Waiting... and width Fill parent. Add Clock1 from Sensors, TimerEnabled true and TimerInterval 200 ms. Set BluetoothClient1.DelimiterByte to 10 (newline).

In Clock1.Timer, if BluetoothClient1.IsConnected and call BluetoothClient1.BytesAvailableToReceive > 0, set Label_PiResponse.Text to call BluetoothClient1.ReceiveText(-1). ReceiveText(-1) reads until the delimiter. Example Pi response: ACK:START followed by newline. The assistant cautioned that BytesAvailableToReceive > 0 does not guarantee a full newline-terminated message has arrived and said this would be improved during Pi testing.

It also corrected its earlier block-name guidance: use Text > unichar with numeric 10 and join it to outgoing messages. Asked to build the Clock receive blocks first.

Exchange 14

User prompt (verbatim)

I found the multiline thing and was able to press enter so i think were good

Assistant answer (condensed transcription)

The assistant said an actual Enter line break in a multiline text block should produce the newline terminator. It advised checking START, STOP, and DIR:CW / DIR:CCW, using a separate multiline text block after the Direction variable when joined dynamically. It emphasized the Enter must create a real second line, not just visual wrapping. Asked whether Clock1.Timer receive blocks were finished.

Exchange 15

User prompt (verbatim)

yeah i think so. So now i just need to handle the pi code which i think ill do in a couple hours

Assistant answer (condensed transcription)

The assistant summarized that the app had START, STOP, Direction, Bluetooth Connect/Disconnect, and prepared receive blocks. Next Pi tasks: Bluetooth setup and pairing; Bluetooth Classic server in C; print received START, STOP, DIR:CW/CCW; send acknowledgments such as ACK:START; test disconnection and reconnection. It noted CP2 did not require integrating motor control yet and that actual Bluetooth communication had not been verified. Suggested returning with 'I'm at the Pi, let's set up Bluetooth'.